Basic scalability — Switch It

Discussion of the **current MVP**, not an imaginary enterprise architecture.

Expected MVP scale

Realistic expectation for this project:

- **Tens to low hundreds** of concurrent users in a limited geographic area.
- Short-lived listings (minutes), so row growth is manageable if expired/completed rows accumulate gradually.
- Not designed as a nationwide real-time parking marketplace.

At this scale, Vercel + Supabase managed Postgres is a reasonable fit.

Database queries that can become expensive

Query / path	Why it can hurt
Map discovery: `parking_spots` where `status = 'available'` and `expires_at > now`	Loads candidate spots for the map; **no PostGIS radius filter** in the listing query
Realtime fan-out on busy `parking_spots` / `claims`	Many clients subscribed while spots churn
History RPC `get_handoff_history`	Safer with keyset pagination; still grows with user history
Claim distance Haversine inside `claim_spot`	Cheap per call; expensive only under claim spam
Live-location Edge Function + Broadcast + snapshot upsert	Extra HTTP + DB writes + Realtime messages while handoffs are active (web and native)

Geographic filtering for discovery is **not** currently a spatial index query. Distance is enforced primarily at **claim** time (1500 m).

Indexes (actually present)

From migrations (notably `20260802110120_initial_schema.sql` and push migration):

Index / unique	Purpose
`parking_spots_status_expires_at_idx`	Speed available/expiry lookups
`parking_spots_owner_created_at_idx`	Owner-related lookups
`parking_spots_one_open_per_owner` (unique partial)	One open listing per publisher
`claims_one_active_per_spot` (unique partial)	Concurrency safety for claims
`claims_one_active_per_seeker` (unique partial)	One active claim per seeker

`claims_seeker_claimed_at_idx`	Seeker claim listing
`claims_spot_status_idx`	Spot↔claim status lookups
`credit_tx_one_debit_per_claim` / `credit_tx_one_credit_per_claim`	Ledger idempotency
`credit_transactions_user_created_at_idx`	User ledger reads
`push_devices_user_enabled_idx`	Optional push device lookup
`handoff_notification_events_pending_idx`	Optional push outbox processing

Not currently implemented: PostGIS / geography indexes for map discovery.

Concurrency

STUDY PRIORITY

Two seekers claiming the same spot:

1. `claim_spot` selects the spot `FOR UPDATE` (serializes claimants).
2. Status must still be `available`.
3. Insert into `claims` with partial unique index `claims_one_active_per_spot`.
4. Unique violations map to business errors.
5. Spot update to `claimed` only if still `available`.

Result: at most one successful active claim. This is **database-enforced**, not React-enforced.

Pagination

Feature	Mechanism
History	Keyset pagination via `get_handoff_history` (`p_limit`, `p_before_at`, `p_before_id`); page size **20**
Map discovery	**Not paginated** by geo tiles; loads current available set

History pagination is the clear “correct pagination” example for the assignment.

Avoiding unnecessary data loading

Meaningful examples from the repo:

- History loads one page at a time (+1 row to detect `hasMore`) — avoids full-history scans.
- Counterpart vehicle fetched only for an active claimed handoff, not for every map pin.
- Realtime tombstones avoid resurrecting terminal spots from stale RSC payloads.
- Live-location traffic is scoped to **active claim** private topics, not global GPS broadcast. Updates are rate-limited in Postgres (2s) before Broadcast.

- The PWA service worker does **not** cache `/_next/` or RSC payloads, so it does not multiply stale hashed-asset traffic.
- Claim button requests a fresh location on submit rather than continuously for the default sheet.

Client / server separation

On client	On server / DB
Map rendering, sheets, countdowns	Auth session, Zod validation
UX distance hints	Authoritative claim distance + locks
Hide illegal buttons	RLS + RPC authorization
Optimistic Realtime merge	Canonical row state

Business invariants for exclusive claims and credit transfers stay in PostgreSQL.

Realtime scaling considerations

Honest limits:

- Every seeker subscribed to spot changes increases Realtime load.
- Dense cities with many short listings increase event volume.
- Missed events are mitigated by refresh/reconcile, but that adds request load.
- Private live-location channels scale with **active handoffs**, not all users.

External service scaling

Service	Note
MapTiler	Tile/style/geocode quotas; key is public in browser
CarImages	Catalog image loader; fallback illustration if unavailable
Supabase	Connection / Realtime quotas on plan
Vercel	Serverless/SSR limits under burst traffic

Push-related Edge/outbox load is **optional/experimental** and not part of the verified core web MVP.

Current bottlenecks / limitations

1. Non-spatial discovery query (fine for MVP, weak at city scale).
2. Realtime + frequent short-lived rows.
3. GPS/location permission UX friction.
4. Expiry often advanced when users hit pages/actions (on-read reconcile), not a dedicated large-scale job system.

5. Database growth of terminal spots/claims over time without archival strategy.

Future improvements (NOT CURRENTLY IMPLEMENTED)

- PostGIS / geohash / bounding-box discovery.
- Server-side clustering for map pins.
- Dedicated expiry worker / queue.
- Read replicas or caching for discovery.
- Rate limiting / abuse detection for claim spam.
- Redis or similar — **not in repo today**.

Repository references

- `src/app/map/page.tsx`
- `src/lib/history/load-history.ts`, `src/lib/history/format.ts`
- `src/lib/map/distance.ts`
- `supabase/migrations/20260802110120_initial_schema.sql`
- `supabase/migrations/20260819130000_prevent_seeker_reclaim.sql`
- `supabase/migrations/20260818180000_handoff_history_page.sql`
- `package.json`